CEL ZAJĘĆ

- Wprowadzenie do języka asembler
- Historia, klasyfikacja, przegląd rozwiązań
- Assembly w porównaniu do innych języków
- rejestry
- Przedstawienie zastosowań języka asembler
- Wady i zalety języka asembler
- Uruchomienie pierwszego programu

ZASADY OCENIANIA

- **JEDNO ZALICZENIE PISEMNE**
- NA KONIEC KAŻDEJ PREZENTACJI PODAWANE BĘDĄ KLUCZOWE POJĘCIA Z ZAJĘĆ

- **REALIZACJA PROJEKTU PRAKTYCZNEGO**
- NA KOLEJNE ZAJĘCIA TJ. 23.03.2023 PROSZĘ O PRZYGOTOWANIE PROPOZYCJI
- PROJEKT WRAZ Z HISTORIĄ ZMIAN POWINIEN ZNALEŹĆ SIĘ W SERWISIE GITHUB
- DOSTARCZENIE PROJEKTU DROGĄ MAILOWĄ
- NAZWY ZMIENNYCH W JĘZYKU ANGIELKSIM + SELF DESCRIPTIVE
- DBAŁOŚĆ O FORMATOWANIE KODU

15. ZASADY ZALICZANIA PRZEDMIOTU:

Zaliczenie na ocenę: test wiedzy oraz zadanie praktyczne.

TREŚCI KSZTAŁCENIA
NARZĘDZIA

11. TREŚCI KSZTAŁCENIA:

1. Klasyfikacja języków - języki maszynowe. Historia powstania języka Assembler.
2. Przegląd rozwiązań sprzętowych i różnych koncepcji assemblera.
3. Podstawowe polecenia (opcode) wybranego procesora
4. Optymalizacja kodu poprzez weryfikację działania poszczególnych opcode.
5. Sposoby adresacji i przekazywania danych pomiędzy oprogramowaniem a sprzętem.
6. Programowanie dodatkowych rejestrów, techniki zaawansowane (łączenie rejestrów, wykorzystanie rejestru flag, sposoby zrównoleglania kodu).
7. Współczesne wykorzystanie kodu niskopoziomowego w programowaniu wysokopoziomowym – tworzenie bibliotek, łączenie aplikacji assembler z językami wysokiego poziomu.

PRZYDATNE LINKI

- https://goalkicker.com/
- https://github.com/EbookFoundation/free-programming-books
- https://www.ibm.com/docs/en/aix/7.3?topic=aix-assembler-language-reference
- https://www.ibm.com/docs/pl/cics-ts/beta?topic=applications-developing-assembler-language
- https://www.mycompiler.io/pl

KLASYFIKACJA JĘZYKÓW
JĘZYKI MASZYNOWE

KLASYFIKACJA:

- Kompilowalne i interpretowalne
- Paradygmat programowania
- Niskiego i wysokiego poziomu
- C i C++ może być kompilowany do assemblera

JĘZYK MASZYNOWY:

- Najniższy poziom programowania
- Sekwencje i instrukcje binarne
- Wykonywany przez CPU
- Charakterystyczny dla architektury procesora

B8 05 00 00 00 // Mov eax, 5 Untitled-2

```
B8 05 00 00 00  // Mov eax, 5
83 C0 0A        // Add eax, 10
C3              // Ret
```

section .text Untitled-3

```
section .text
global _start

_start:
    mov eax, 5      ; Przeniesienie wartości 5 do rejestru eax
    add eax, 10     ; Dodanie 10 do wartości w eax
    ; Wynik znajduje się teraz w rejestrze eax
    ; Dodatkowe operacje lub zakończenie programu
    ; ...

    ; Instrukcja zakończenia programu
    mov ebx, eax    ; Przeniesienie wyniku do rejestru ebx
    mov eax, 1      ; Kod wyjścia
    int 0x80        ; Wywołanie przerwania systemowego
```

Języki Maszynowe	Języki Assemblera
Języki maszynowe to binarna reprezentacja instrukcji zrozumiała dla procesora.	Języki assemblera to symboliczna reprezentacja kodu maszynowego.
Instrukcje te są bezpośrednio wykonywane przez jednostkę centralną CPU.	Zamiast używania surowych liczb binarnych, korzystają z mnemoników i symboli dla lepszej czytelności.
Języki maszynowe są trudne do zrozumienia i edycji przez ludzi.	Języki assemblera są bardziej abstrakcyjne i przenośne, ponieważ są tłumaczone na konkretne języki maszynowe przez assembler.
10110010 00101100	**MOV AL, 2C**
Języki assemblera stanowią pewne ułatwienie dla programistów, umożliwiając pracę na niższym poziomie abstrakcji bez konieczności korzystania bezpośrednio z kodu maszynowego.	

ASSEMBLER - HISTORIA POWSTANIA

- Asembler jako najwcześniejszy język programowania niskiego poziomu pojawił się w latach 40.
- Lata 50-60: Rozwój powiązany z architekturami
- Lata 70: Standardyzacja
- Lata 80: Popularność komputerów osobistych
- Lata 90: Zdobywanie popularności języków wysokiego poziomu.
- Asembler traci popularność wśród programistów aplikacji, pozostaje używany w obszarze systemów operacyjnych, sterowników, i programowania niskiego poziomu.

Współczesność:

- XXI wiek: Asembler nadal istnieje w obszarze optymalizacji kodu, programowania niskopoziomowego, tworzenia sterowników i systemów wbudowanych.
- Wprowadzenie nowych asemblerów, takich jak dla architektury ARM w kontekście urządzeń mobilnych.

Wykorzystanie w Edukacji:

- Asembler jest często używany w edukacji informatycznej do zrozumienia działania procesora na poziomie maszynowym.
- Pomaga zrozumieć koncepcje takie jak rejestry, adresowanie i operacje logiczne.

Przyszłość Asemblera:

- Nadal istotny w obszarze optymalizacji i niskopoziomowego programowania.
- Przy rosnącej złożoności układów, asembler pozostanie narzędziem używanym przez zaawansowanych programistów.

PRZEGLĄD ROZWIĄZAŃ I RÓŻNYCH KONCEPCJI ASSEMBLERA

	Architektura	Zastosowanie
x86 Assembly	32- i 64-bitowe wersje, stosowane w komputerach osobistych i serwerach.	Powszechnie używana w systemach operacyjnych, oprogramowaniu aplikacyjnym. Dominująca w przemyśle komputerowym.
ARM Assembly	Stosowana w mikrokontrolerach i urządzeniach mobilnych, charakteryzuje się niskim zużyciem energii.	Powszechna w smartfonach, tabletach, systemach wbudowanych. Znajduje zastosowanie w IoT.
MIPS Assembly	Wykorzystywana w systemach wbudowanych, routerach, konsolach do gier.	Powszechna w systemach wbudowanych, gdzie wymagana jest stabilność i wydajność.
SPARC Assembly	Architektura RISC (Reduced Instruction Set Computing).	Stosowana w serwerach, stacjach roboczych, systemach Sun Microsystems.
PowerPC Assembly	Popularna w komputerach Mac i niektórych konsolach do gier.	Wykorzystywana w niektórych komputerach Mac, konsolach gier.
NASM	Assembler ogólnego przeznaczenia. Netwide assembler, 1996	Uniwersalny, umożliwiający pisanie kodu dla różnych architektur procesorów. Często używany w edukacji, projektach niskopoziomowych.
FASM,GAS	Wieloplatformowy, często używany w systemach opartych na architekturze x86 i ARM.	Niskopoziomowe programowanie na różnych platformach. Tworzenie aplikacji dla systemów wbudowanych.

ASEMBLER

- Assembler to język programowania niskiego poziomu, który operuje na poziomie bezpośrednio związanych z architekturą komputera instrukcji procesora. Jest stosowany tam, gdzie wymagana jest precyzyjna kontrola nad sprzętem, co czyni go narzędziem idealnym do programowania systemowego i optymalizacji kodu.
- Assembler pełni kluczową rolę w procesie tłumaczenia kodu źródłowego na kod maszynowy. Dzięki assemblerowi programista ma bezpośrednią kontrolę nad rejestrami, pamięcią i innymi zasobami sprzętowymi.
- Assembler jest dostosowywany do konkretnych architektur procesora. Przykładowe architektury obejmują x86 (Intel, AMD), ARM (wielu producentów, w tym Apple i Qualcomm) oraz MIPS (często stosowany w systemach wbudowanych).

hello.asm

lab1c1 > hello.asm

```
section .data
    msg db      "witaj, świecie",0
section .bss
    buffer resb 256
section .text
    global main
main:
    mov     rax, 1
    mov     rdi, 1
    mov     rsi, msg
    mov     rdx, 15
    syscall

    mov     rax, 60
    mov     rdi, 0
    syscall
```

NASM
Netwide Assembler

REJESTRY

- Rejestry są małymi obszarami pamięci znajdującymi się bezpośrednio w procesorze.
- Używane do przechowywania danych tymczasowych i wykonywania operacji.
- Rejestry ogólnego przeznaczenia (GPR), do użycia przez programistę, np.: EAX, EBX, ECX, EDX
- Rejestry specjalne:
- Rejestr instrukcji (EIP) - zawiera adres bieżącej wykonywanej instrukcji.
- Rejestr stosu (ESP) - Używany do przechowywania wskaźnika stosu.
- Rejestr bazowy (EBP) - Często używany jako wskaźnik na stos w ramach funkcji.

Operacja	**Przykład**
Przypisanie Wartości	MOV EAX, 10
Dodawanie	ADD EBX, EAX
Odejmowanie	SUB ECX, EDX
Mnożenie	IMUL ESI, EDI, 2

URUCHOMIENIE

Makefile służy do automatyzacji budowania aplikacji.
Linker wykonuje konsolidację czyli przetwarza kod i wyszukuje w systemie inne potrzebne pliki, zwykle usługi systemowe. Następnie pliki te są konsolidowane z wygenerowanym kodem w celu utworzenia pliku wykonywalnego.

makefile

```
hello: hello.o
    gcc -o hello hello.o -no-pie
hello.o: hello.asm
    nasm -f elf64 -g -F dwarf hello.asm -l hello.lst
```

LINKER

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash

```
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY$ make
nasm -f elf64 -g -F dwarf hello.asm -l hello.lst
gcc -o hello hello.o -no-pie
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY$ ./hello
witaj, świecieahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY
ahnsuser@Ubuntu-AHNS-Prog-Niskopoziomowe:~/Desktop/assembly/prog_niskopoziomowe/imie_nazwisko_DD_MM_YYYY$
```

ZASTOSOWANIE

- Programowanie wbudowane
- Systemy operacyjne
- Sterowniki sprzętu
- Programowanie na platformy o ograniczonych zasobach
- Optymalizacja kodu
- Programowanie niskopoziomowe
- Tworzenie programów diagnostycznych
- Programowanie sterowników grafiki i gier
- Kryptografia

ZALETY I WADY

- Pełna kontrola nad sprzętem
- Niska warstwa abstrakcji
- Optymalizacja kodu
- Mała ilość pamięci i zasobów
- Nauka architektury procesora

- Złożoność i długość kodu
- Brak przenośności
- Trudności w rozwoju i utrzymaniu
- Ryzyko błędów
- Ograniczone zasoby biblioteczne
- Długi czas rozwoju

```python
def isPrime(n):
    for i in range(2,n//2+1):
        if (not (n%i)):
            return 0;
    return 1;

numPrimes=0;
for i in range(2, 250001)
    numPrimes+=isPrime(i);

print(str(numPrimes));
```

```c
#include <stdio.h>
int isPrime(int n){
    for (int i=2;i<=n/2;i++)
        if (!(n%i))
            return 0;
    return 1;
}


void main(){
    int numPrimes=0;

    for (int i = 2;i<250001;i++)
        numPrimes+=isPrime(i);

    printf("%d\n",numPrimes);
}
```

```asm
.section .data
    f: string "%d\n"
.section .text
    .global main
    main:
        movl $2, %eax
        xor %r8d,%r8d
    loop:
        cmpl $250000,%eax
        jg end_loop
        movl $2,r10d
        movl %eax,r11d
        shr $1,%r11d
    prime_loop:
        cmpl %r11d, %r10d
        jg prime
        push %rax
        xor %edx,%edx
        div %r10d
        test %edx,%edx
        pop %rax
        je not_prime
        inc %r10d
        jmp prime_loop
    prime:
        inc %r8d
    not_prime:
        inc %eax

    end_loop:
        mov %r8d,%esi
        xor %eax,%eax
        call printf
```

	PYTHON	C	X86 ASM
Czas wykonywania [s]	54,263	5,302	5,307

KLUCZOWE POJĘCIA

- Język wysokiego poziomu vs język niskiego poziomu + przykłady
- Assembler opis
- Assembler zastosowanie
- Assembler wady i zalety
- Linker
- Rejestry